# Day 116 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 17, Day 5 reference

Watch: 7.7 Stacking (L07: Ensemble Methods) (Sebastian Raschka) | Stacking Explained | Advanced Ensemble Learning in Machine Learning (Tech Pulse Labs)

`day116_stacking_blending.py`, segment by segment, in the order `main()` uses things. Softmax regression, the five base-model wrappers, out-of-fold predictions, stacking and blending are new; the models inside the wrappers come unchanged from Days 107, 109, 112, 113 and 115. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

**MATH BOX LEGEND**

**Symbols:** P = predicted probabilities, one row per point · F = meta-features · W, b = the meta-model's weights and biases · k = number of classes · Y = one-hot labels

## Segment 0 — Setup

### Imports and settings

**CODE**

```
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE); sys.path.insert(0, os.path.join(HERE, "..",
    "week-16-classical-ml-trees"))
with contextlib.redirect_stdout(io.StringIO()):
    import day113_kernel_trick as d113
    import day115_k_nearest_neighbors as d115
    import day109_gradient_boosting as d109
d112, d107 = d113.d112, d109.d107
standardise, stratified_split, d99 = d112.standardise, d112.stratified_split, d112.d99
FOLDS = 5
SPLITS = {"Breast cancer": 20, "Titanic": 20, "Digits": 5}
EPS = 1e-6
```

- `sys.path.insert(0, os.path.join(HERE, "..", "week-16-classical-ml-trees"))` — lets Python find Week 16's lessons, one folder over.
- `d112, d107 = d113.d112, d109.d107` — earlier lessons reached through the ones that imported them: no second copy.
- `SPLITS = {"Breast cancer": 20, ...}` — fewer splits for digits, the slowest problem.

## Segment 1 — The meta-model

### softmax and fit_softmax

**MATH**

$$P = \text{softmax}(FW + b); \quad \text{gradient at the scores} = (P - Y)/n; \quad dW = F^T(P - Y)/n + \lambda W$$

**CODE**

```
def softmax(Z):
    Z = Z - Z.max(1, keepdims=True); E = np.exp(Z); return E / E.sum(1, keepdims=True)
def fit_softmax(F, y, k, lam=1e-3, steps=500, lr=0.5):
    W = np.zeros((F.shape[1], k)); b = np.zeros(k); Y = np.eye(k)[y]
    m = [np.zeros_like(W), np.zeros_like(b)]; v = [np.zeros_like(W), np.zeros_like(b)]
    for t in range(1, steps + 1):
        G = (softmax(F @ W + b) - Y) / len(y)
        for i, (p, g) in enumerate(((W, F.T @ G + lam * W), (b, G.sum(0)))):
            m[i] = 0.9 * m[i] + 0.1 * g; v[i] = 0.999 * v[i] + 0.001 * g * g
            p -= lr * (m[i] / (1 - 0.9 ** t)) / (np.sqrt(v[i] / (1 - 0.999 ** t)) + 1e-8)
    return W, b
```

- `Z - Z.max(1, keepdims=True)` — subtracting each row's maximum keeps exp from overflowing; the result is unchanged.
- `np.eye(k)[y]` — one-hot labels: row c of the identity matrix for class c.
- `for i, (p, g) in enumerate(((W, ...), (b, ...)))` — one Adam loop over two (parameter, gradient) pairs.
- `p -= ...` — in place, so W and b themselves change.

## Segment 2 — Base models

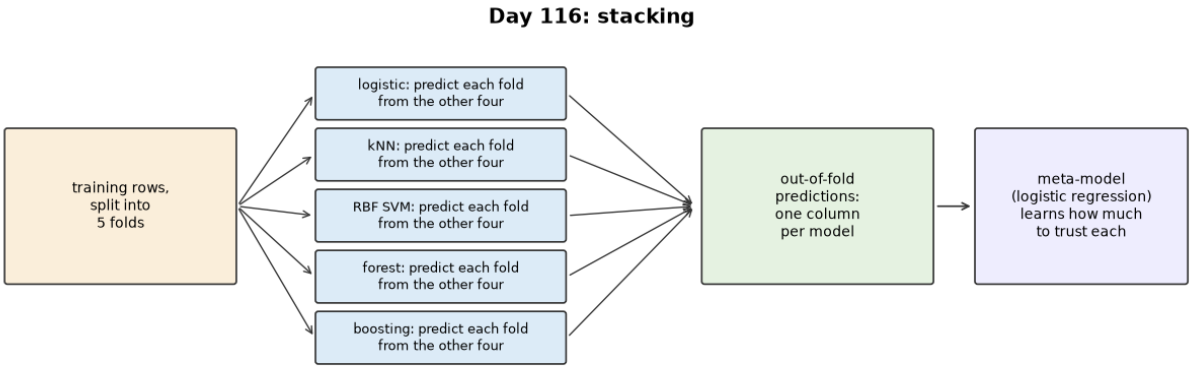
### The base models as fit(X, y, k, seed) -> proba

CODE

```
def m_logistic(X, y, k, seed):
    mu, sd = X.mean(0), X.std(0) + 1e-12; W, b = fit_softmax((X - mu) / sd, y, k)
    return lambda Z: softmax(((Z - mu) / sd) @ W + b)
def m_knn(X, y, k, seed, nn=15):
    mu, sd = X.mean(0), X.std(0) + 1e-12; A = (X - mu) / sd
    def proba(Z):
        D = d115.sqdist((Z - mu) / sd, A); idx = np.argpartition(D, nn - 1, axis=1)[:, :nn]
        return np.stack([np.mean(y[idx] == c, 1) for c in range(k)], 1)
    return proba
def m_svm(X, y, k, seed, C=1.0):
    mu, sd = X.mean(0), X.std(0) + 1e-12; A = (X - mu) / sd; g = 1.0 / A.shape[1]
    K = d113.rbf(A, A, g); cls = [1] if k == 2 else range(k)
    fits = [d113.fit_kernel_svm(K, np.where(y == c, 1.0, -1.0), C)[:2] for c in cls]
    def proba(Z):
        Kz = d113.rbf(A, (Z - mu) / sd, g)
        S = np.stack([Kz.T @ (a * np.where(y == c, 1.0, -1.0)) + bb for c, (a, bb) in zip(cls,
            fits)], 1)
        return softmax(np.hstack([-S / 2, S / 2])) if k == 2 else softmax(S)
    return proba
def m_forest(X, y, k, seed, trees=60):
    f, _ = d107.fit_forest(X, y, k, trees, max(1, int(np.sqrt(X.shape[1]))), seed)
    return lambda Z: d107.forest_proba(f, Z, k)
def m_boost(X, y, k, seed):
    m, _ = d109.fit_boost(X, y, "logistic", 150, 0.1, 3)
    def proba(Z):
        F = np.full(len(Z), m["base"])
        for t in m["trees"]:
            F = F + m["lr"] * d109.predict_reg(t, Z)
        p = d109.sigmoid(F); return np.stack([1 - p, p], 1)
    return proba
BASE = {"logistic": m_logistic, "kNN": m_knn, "RBF SVM": m_svm, "forest": m_forest, "boosting":
    m_boost}
```

- `def proba(Z): ... return proba` — each wrapper returns a function that remembers its training data: a closure.
- `np.stack([np.mean(y[idx] == c, 1) for c in range(k)], 1)` — kNN's probabilities: the share of the 15 neighbours in each class.
- `softmax(np.hstack([-S / 2, S / 2]))` — two-class SVM scores squashed into shares; the script's comment calls it a rough choice.
- `BASE = {name: function}` — a dictionary of fit functions, so every experiment loops over the same models.

SYSTEM DIAGRAM




*Out-of-fold predictions feed the meta-model.*

## Segment 3 — Stacking and blending

### features and oof

MATH

$oof_i$ = *model trained without fold(i), applied to* $x_i$;  *log-odds* = $log(p_1/p_0)$

CODE

```
def features(P, k):
    P = np.clip(P, EPS, 1 - EPS)
    return np.log(P[:, 1] / P[:, 0])[:, None] if k == 2 else np.log(P)
def oof(fit, X, y, k, seed):
    folds = np.array_split(np.random.RandomState(seed).permutation(len(y)), FOLDS); P =
        np.zeros((len(y), k))
    for i in range(FOLDS):
        va = folds[i]; tr = np.concatenate([folds[j] for j in range(FOLDS) if j != i])
        P[va] = fit(X[tr], y[tr], k, seed)(X[va])
    return P
```

- `np.clip(P, EPS, 1 - EPS)` — keeps probabilities off 0 and 1, so the log is finite.
- `np.log(P[:, 1] / P[:, 0])[:, None]` — log-odds as a column: [:, None] adds the second axis.
- `P[va] = fit(X[tr], y[tr], k, seed)(X[va])` — the fold's rows are filled by a model trained without them.

### stack and blend

CODE

```
def stack(names, X, y, Xte, k, seed, leak=False):
    fits = {n: BASE[n](X, y, k, seed) for n in names}
    tr = [features(fits[n](X) if leak else oof(BASE[n], X, y, k, seed), k) for n in names]
    te = [features(fits[n](Xte), k) for n in names]
    W, b = fit_softmax(np.hstack(tr), y, k)
    return softmax(np.hstack(te) @ W + b), W, {n: fits[n](Xte) for n in names}
def blend(names, X, y, Xte, k, seed, frac=0.3):
    rng = np.random.RandomState(seed); o = rng.permutation(len(y)); h = int(len(y) * frac); ho,
        fit_rows = o[:h], o[h:]
    fits = {n: BASE[n](X[fit_rows], y[fit_rows], k, seed) for n in names}
    W, b = fit_softmax(np.hstack([features(fits[n](X[ho]), k) for n in names]), y[ho], k)
    return softmax(np.hstack([features(fits[n](Xte), k) for n in names]) @ W + b)
```

- `features(fits[n](X) if leak else oof(...), k)` — the one switch between the honest version and the leakage trap.
- `np.hstack(tr)` — each model's features side by side: one row per point, one block of columns per model.
- `o[:h], o[h:]` — blending's split: the first 30% of a shuffled order for the meta-model, the rest for the base models.

## Segment 4 — The experiments

### demo_compare: the scoreboard

CODE

```
for s in range(SPLITS[name]):
    tr, te = stratified_split(y, 0.3, s)
    t0 = time.perf_counter(); P, _, base = stack(names, X[tr], y[tr], X[te], k, s);
        t_stack.append(time.perf_counter() - t0)
    singles = {n: acc(base[n], y[te]) for n in names}
    cv = {n: acc(oof(BASE[n], X[tr], y[tr], k, s), y[tr]) for n in names}
    avg = acc(np.mean([base[n] for n in names], 0), y[te])
    rows.append([singles[n] for n in names] + [singles[max(cv, key=cv.get)], avg, acc(blend(names,
        X[tr], y[tr], X[te], k, s), y[te]), acc(P, y[te])])
```

- `time.perf_counter()` — a high-resolution clock around the stack call.
- `max(cv, key=cv.get)` — the base model with the best out-of-fold accuracy on the training rows: the choice without stacking.
- `np.mean([base[n] for n in names], 0)` — the simple average of the models' probabilities.

### demo_learned: how alike are the mistakes?

CODE

```
w = np.mean(Ws, 0); E = np.hstack(E).astype(float)
print("Titanic, 20 splits: mean meta-weight on each model's log-odds: " + ", ".join(f"{n}
    {x:+.3f}" for n, x in zip(names, w)))
C = np.corrcoef(E)
print(f"\ncorrelation between models' mistakes (1 = the same rows wrong):\n  {'':<10}" +
    "".join(f"{n:>10}" for n in names))
for i, n in enumerate(names):
    print(f"  {n:<10}" + "".join(f"{C[i, j]:>10.3f}" for j in range(len(names))))
allwrong = np.mean(E.min(0) == 1); onewrong = np.mean((E.sum(0) > 0) & (E.min(0) == 0))
```

- `np.corrcoef(E)` — the correlation between every pair of rows of E: one row per model, 1 where it was wrong.
- `E.min(0) == 1` — a test row every model got wrong.

- `(E.sum(0) > 0) & (E.min(0) == 0)` — a row some model got wrong and some got right: the only rows a combination can fix.

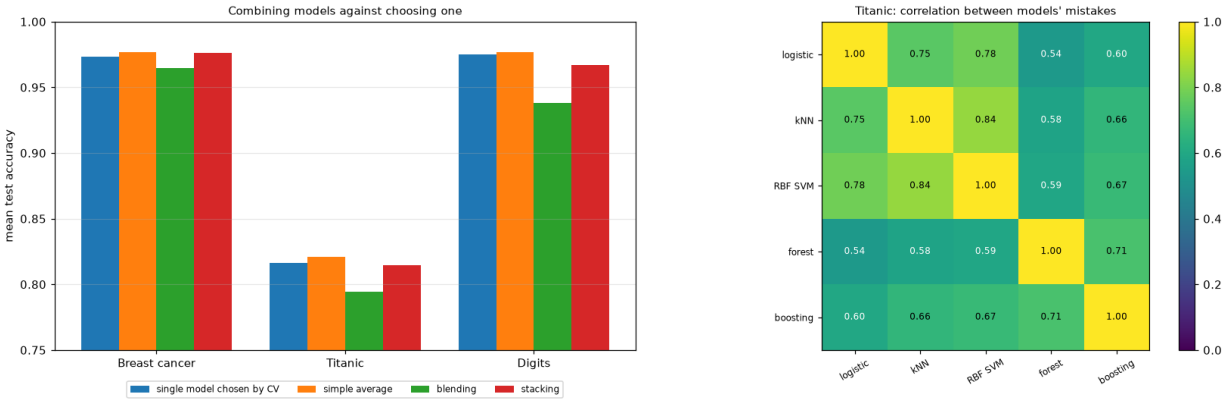
### demo_diversity: temporarily adding models

**CODE**

```
BASE.update({"forest A": lambda X, y, k, s: m_forest(X, y, k, s + 100), "forest B": lambda X, y,
    k, s: m_forest(X, y, k, s + 200)})
for s in range(10):
    tr, te = stratified_split(y, 0.3, s); row = []
    for group in (["forest", "forest A", "forest B"], ["forest", "logistic", "kNN"]):
        P, _, base = stack(group, X[tr], y[tr], X[te], 2, s)
        row += [acc(P, y[te]), max(acc(base[n], y[te]) for n in group)]
    rows.append(row)
BASE.clear(); BASE.update(saved)
```

- `saved = dict(BASE)` — a copy of the dictionary, taken before changing it.
- `lambda X, y, k, s: m_forest(X, y, k, s + 100)` — the same forest with a different seed: a near-copy.
- `BASE.clear(); BASE.update(saved)` — restores the five base models, so later code is unaffected.

**REAL GRAPH**




*Real graph from the script: the approaches compared, and the mistake correlations.*

## Quick Syntax Glossary — new or reinforced constructs

- `Z - Z.max(1, keepdims=True)` — a stable softmax.
- `np.eye(k)[y]` — one-hot labels.
- `def f(): ... return inner` — a closure: a function that remembers its data.
- `x[:, None]` — a column vector.
- `np.corrcoef(E)` — correlations between rows.
- `dict(d); d.clear(); d.update(saved)` — copy, empty and restore a dictionary.

*End of Day 116 syntax reference. For the experiments and what they showed, see the Day 116 complete guide.*